2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) | 979-8-3315-3510-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/SANER64311.2025.00037

Boosting Large Language Models for System Software Retargeting: A Preliminary Study

Ming Zhong[12], Fang Lv*[1], Lulin Wang[1], Lei Qiu[12], Hongna Geng[12], Huimin Cui*[12], Xiaobing Feng[12]
[1]SKLP, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
[2]University of Chinese Academy of Sciences, Beijing, China
{zhongming21s, flv, wanglulin, qiulei21b, genghongna, cuihm, fxb}@ict.ac.cn

***Abstract*—System software bridges hardware platforms and high-level applications. As new hardware platforms emerge, developers must customize code to support various system software, a process known as "retargeting". This process is time-consuming and poorly automated. While large language models (LLMs) are proficient in general code generation tasks, their effectiveness in retargeting is limited by code complexity and abstract function descriptions. This paper presents TeSyn, a novel framework to enhance the code generation capabilities for system software retargeting. TeSyn comprises three steps: target-specific value extraction, common code clustering, and template synthesis. To evaluate TeSyn's effectiveness, we introduce SysRetar, the first dataset for system software retargeting, covering four types of system software and 195 hardware platforms. In our experiments, we select five LLMs and fine-tune CodeLLaMA-7B-Instruct on SysRetar to create SysRetar-LLM. Results show that TeSyn significantly enhances retargeting performance across five LLMs. Furthermore, code generated by SysRetar-LLM requires substantially less modification than the manual retargeting approach (Fork-Flow), suggesting potential improvements in efficiency. Given these promising results, we outline future research directions for advancing retargeting through LLMs. The dataset and code are publicly available at https://huggingface.co/docz1105/SysRetar-LLM.**

***Index Terms*—Software Retargeting, LLMs, Code Generation.**

I. Introduction

With the diversification of application scenarios, specialized hardware platforms such as GPUs have emerged. To ensure seamless interaction with these platforms (hereafter referred to as "**targets**"), developers write code for various system software—including operating systems, compilers, hypervisors, etc [24]—through a process known as "retargeting" [5], [32].

Modular design allows developers to write only the necessary code to support the unique features of new targets during retargeting, avoiding the need to recreate the entire system software. For example, compiler developers focus solely on building the backend for a new target, bypassing frontend and middle-end development. In the retargeting process, developers provide target-specific implementations for functions that handle characteristics like instructions and registers. These functions are generally categorized as **inherited** and **customized**. **Inherited functions** are standard functions with unified interfaces provided by the system software, from which developers select and implement a subset based on target-specific requirements, potentially reappearing across multiple targets. **Customized functions**, however, are unique to a single target, supporting its distinct features.

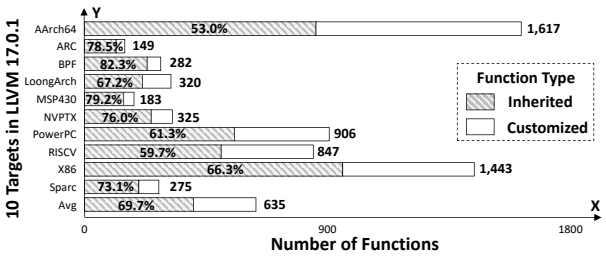


Fig. 1: Proportion of functions in backends development for 10 targets in LLVM 17.0.1.

Despite modularity and unified inherited function interfaces, developers still need to write a significant amount of code, making the process time-consuming. In LLVM 17.0.1, a mainstream compiler infrastructure [17], data from 10 targets (Fig. 1) shows that creating a backend typically involves writing 635 functions, averaging 32.1 KLoC (Kilo lines of code) in C/C++. Of these, 69.7% are inherited functions and 30.3% are customized, reflecting the substantial effort.

In the AI field, pre-trained models are widely used in code generation [4], [9], [10], [29], particularly large language models (LLMs) that can swiftly generate code based on natural language descriptions [11], [19], [21], [25], [27], significantly reducing manual effort.

Inspired by advancements in code generation using LLMs, we propose leveraging them for retargeting code generation. However, as demonstrated in Sec. V-B, directly prompting state-of-the-art LLMs (e.g., ChatGPT) is ineffective due to the significant variations in hardware characteristics across targets. Furthermore, relying solely on user-provided natural language descriptions limits the LLMs' ability to accurately map these inputs to specific functionalities in system software.

In this work, we propose TeSyn, a Template Synthesis framework to enhance LLMs for retargeting code generation. Our key insight is to extract common code from functions with identical functionalities across different targets, synthesize function templates, and incorporate these into LLM prompts. This reduces code variations and establishes a clear mapping between functional descriptions and code. Additionally, we introduce SysRetar, the first dataset for system software retar-

*: Corresponding Author.

2640-7574/25/$31.00 ©2025 IEEE
DOI 10.1109/SANER64311.2025.00037

Authorized licensed use limited to: William & Mary. Downloaded on October 10,2026 at 03:41:57 UTC from IEEE Xplore. Restrictions apply.

```
P1 const FrameLowering *TFI = getFrameLowering(MF);
P2 markSuperRegs(Reserved, PPC::ZERO);
```
(a) PowerPC

```
A1 const FrameLowering *TFI = getFrameLowering(MF);
A2 markSuperRegs(Reserved, AArch64::WSP);
A3 ...
A4 if (TFI->hasFP(MF)) markSuperRegs(Reserved, AArch64::W29);
```
(b) AArch64

```
L1 const FrameLowering *TFI = getFrameLowering(MF);
L2 markSuperRegs(Reserved, LoongArch::R0);
L3 ...
L4 if (TFI->hasBP(MF)) markSuperRegs(Reserved, LoongArch::R31);
```
(c) LoongArch

Fig. 2: Code segments of getReservedRegs function in PowerPC, AArch64 and LoongArch. Note that A_4 and L_4 differ, with A_4 using the **hasFP** (Frame Pointer Register) and L_4 using the **hasBP** (Base Pointer Register), each corresponding to distinct registers.

geting, covering retargeting code for 195 targets across four types of open-source system software.

Preliminary results with five LLMs show that TeSyn significantly improves retargeting code generation accuracy. Fine-tuning CodeLLaMA-7B-Instruct on SysRetar to create SysRetar-LLM also surpasses traditional retargeting methods (Fork-Flow) in accuracy. Based on these promising outcomes, we propose several research directions for further improving retargeting efficiency with LLMs.

II. Motivation

We use the getReservedRegs function in LLVM to illustrate the regularity of implementing inherited functions during the retargeting process. Fig. 2 shows implementations of getReservedRegs for three targets. While the function's description–"Returns a bitset..." – is the same, differences in code arise due to target-specific values like register names. For example, the first statements are identical across all targets, but the blue segments of PowerPC's P_2, AArch64's A_2, and LoongArch's L_2 differ due to each target's unique register naming conventions. These disparities in codes are influenced by the instruction sets and hardware characteristics.

Observations from inherited functions in system software reveal that **target-specific values** fall into three categories: (1) numerical values, (2) strings in double quotes, and (3) enumeration values prefixed by the target's name. This regularity allows target-specific values to be *automatically extracted using fixed patterns*. After removing these target-specific values (blue segment in Fig. 2), the remaining code forms the **common code** (black segment in Fig. 2).

During retargeting, inherited functions across different targets often share substantial common code, which is consistent across various system software. However, some targets require special handling due to unique characteristics, making single-target templates ineffective. Synthesizing identical functions from multiple targets into a single template enhances generalization ability. While most developers follow similar coding styles, certain targets have distinct styles, necessitating code clustering during template synthesis to reduce their impact. **This work focuses only on inherited functions, as customized functions do not fit template synthesis criteria**.

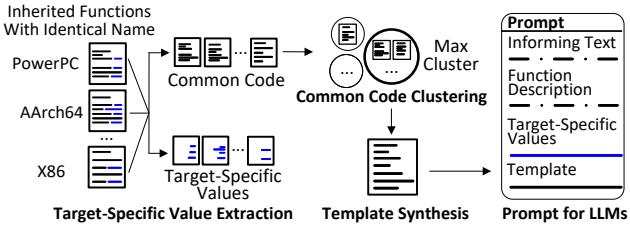


Fig. 3: Workflow of TeSyn.

III. TeSyn: A Template Synthesis Framework

A. Research Question

This work aims to explore the feasibility and effectiveness of LLMs in system software retargeting by addressing two research questions (RQ):

- **RQ.1** Can existing LLMs generate code for system software retargeting, and can prompt construction enhance the accuracy of generated code?
- **RQ.2** Can LLMs outperform traditional manual methods (Fork-Flow) in terms of code accuracy for retargeting?

Answers to these questions will inspire further research in AI and SE community, potentially enhancing the efficiency of manual system software retargeting.

B. Workflow of TeSyn

We begin by automatically grouping functions from different targets with identical names within the system software. As shown in Fig. 3, TeSyn processes each group through three steps: target-specific value extraction, common code clustering, and template synthesis.

During target-specific value extraction, common code and target-specific values are separated into two distinct parts within the functions. Next, common code is clustered to mitigate the impact of highly specific coding styles in certain targets. A function template is then synthesized from the largest cluster for each function. Finally, prompts are generated based on the synthesized templates.

C. An End-to-End Example

Fig. 4 presents an end-to-end example of how TeSyn automatically constructs a prompt for getReservedRegs using implementations from Fig. 2 and other targets. Identically named functions from different targets are grouped. Based on this grouping, we introduce the three key steps in TeSyn.

1) Key Steps: **Target-Specific Value Extraction.** We begin by extracting target-specific values from each function, such as register names in getReservedRegs (P_2, A_2, A_4, L_2, L_4 in Fig. 2). Using fixed patterns (numbers, double quotes, enumerated variables) based on the three categories in Sec. II, we automatically match and extract relevant code, excluding non-target-specific values like loop counters (e.g., cnt = 1). The remaining code after extraction forms the common code.

Authorized licensed use limited to: William & Mary. Downloaded on October 10,2026 at 03:41:57 UTC from IEEE Xplore. Restrictions apply.

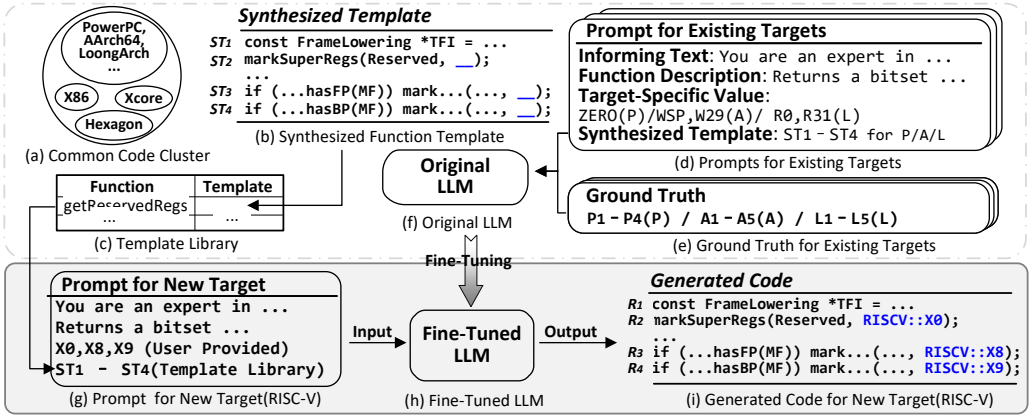


Fig. 4: An end-to-end example of using TeSyn to fine-tune LLMs for code generation, with "P" for `PowerPC`, "A" for `AArch64`, and "L" for `LoongArch`.

Common Code Clustering. To mitigate the impact of specific coding styles on template synthesis, common code across all targets is clustered using the affinity propagation algorithm [6] and "Edit Distance" [26] as the evaluation metric. Clustering is based purely on code structure, akin to text clustering, without considering semantics. The largest cluster is then used for template synthesis, as shown in Fig. 4(a).

Template Synthesis. During template synthesis, common code from the largest cluster is merged using the difflib tool [23] to identify identical and unique statements. Identical statements (e.g., ST_1-ST_2 in Fig. 4(b)) are retained once, while unique statements (e.g., ST_3-ST_4 in Fig. 4(b)) are included directly. This preserves target differences while reducing redundancy. The synthesized templates are then stored in a library for future use, as shown in Fig. 4(c).

2) Prompt Construction: With synthesized template, as shown in Fig. 4(d), TeSyn creates an input prompt with four components: informing text, function description, target-specific values, and the template. The informing text guides the model during code generation.

Informing Text

System: You are an expert in writing LLVM code. Please provide a function with the following function information.
User: Write a function named getReservedRegs for `PowerPC`.

The function description is automatically extracted from official system software documentation, such as the LLVM doxygen document, and code comments. During model fine-tuning, the original code of each function is provided as ground truth to the model, such as A_1-A_4 for `AArch64` in Fig. 4(e).

3) Model Fine-Tuning: During fine-tuning (Fig. 4(f) and Fig. 4(h)), the model is trained for sequence-to-sequence prediction [9], [29]. The constructed prompt serves as input, with the corresponding function code as ground truth.

4) Generation For New Targets: When creating an inherited function for a new target, such as `RISC-V` in Fig. 4(g), users simply provide the function name and its target-specific values. TeSyn automatically retrieves the function's description and template in the library, constructing a prompt accordingly. The fine-tuned LLM (Fig. 4(h)) subsequently generates the function (Fig. 4(i)) with the prompt.

TABLE I: Composition of SysRetar. "Official" refers to the officially released versions of each software, while "GitHub" denotes open-source repositories collected from GitHub. "Targets" represents the number of targets in each software, calculated by merging identical targets from different repositories.

Software	File Path for Retargeting	Data Source	Targets
LLVM	/llvm/llvm/lib/Target/*	Official: 2.0.1 - 17.0.1 GitHub: 296 repositories	101
GCC	/gcc/gcc/config/*	Official: 3.0 - 13.0 GitHub: 21 repositories	77
xvisor	/xvisor/arch/*	Official: 0.1.0 - 0.3.2	3
musl	/musl/arch/*	Official: 1.0.0 - 1.2.5	14

IV. SysRetar : A Dataset for Retargeting

A. System Software

To fine-tune LLMs and assess TeSyn's effectiveness, we constructed a dataset specialized for system software retargeting, named SysRetar. SysRetar consists of four kinds of open-source system software, including two compilers, LLVM [17] and GCC [8], a hypervisor, xvisor [30], and a C language library, musl [20]. They can be used to assess the efficacy of TeSyn across *different types of system software* and *different software (GCC and LLVM) within the same type (compiler)*. The composition of SysRetar is provided in Table I.

B. Data Pre-processing

We extracted all inherited functions from C/C++ files in Table I using tree-sitter [28], removing comments and identifying statement endings by symbols ("{", "}", ";", ":"). Functions with the same name were grouped within the same type of software, containing different target implementations.

C. Dataset Split

We selected 4 targets in LLVM (`RISC-V`, `RI5CY`, `ARC`, and `NVPTX`), 3 in GCC (`RISC-V`, `ARC`, and `NVPTX`), 1 in xvisor (`RISC-V`), and 2 in musl (`RISC-V` and `MicroBlaze`) to form the test set, covering various application scenarios (e.g., CPU, GPU, and Micro-Processor). The remaining targets,

Authorized licensed use limited to: William & Mary. Downloaded on October 10,2026 at 03:41:57 UTC from IEEE Xplore. Restrictions apply.

including 97 in LLVM, 74 in GCC, 2 in xvisor, and 12 in musl, are used for constructing the train and validation sets, effectively preventing data leakage.

We exclude data from all test set targets and split **each function's prompts** from various target implementations into 90% train and 10% validation sets. This ***function-based scheme*** ensures templates appear in both sets, improving fine-tuning. For functions with few implementations, like those in xvisor with only two targets (excluding RISC-V), one prompt is placed in the train set and the rest in validation, making the final train/validation ratio less than 90%:10%. This results in 10,709/2,090/1,416 functions in the train/validation/test sets.

V. Results & Discussion

A. Experiment Setup

Fundamental Models. We select five LLMs to evaluate TeSyn, including ChatGPT-3.5-Turbo [21], CodeLLaMA-7B-Instruct [25], CodeGemma-7B-IT [27], DeepSeek-Coder-6.7B-Instruct [11], Grantie-8B-code-instruct [19]. These five models were directly used for inference testing. Additionally, we fine-tuned the CodeLLaMA-7B-Instruct with SysRetar.

Baselines. We evaluated TeSyn under two prompt scenarios: (1) prompts with informing text, function descriptions, and target-specific values (**–w/o TeSyn**), and (2) prompts generated by TeSyn with a synthesized function template (**-w TeSyn**). Scenario (1) serves as the baseline, and improvements in scenario (2) assess TeSyn's impact on LLM performance.

Evaluation Metrics. To evaluate model inference capabilities, we use the following metrics: Edit Distance (ED) [9], BLEU-4 [22], and CodeBERTScore [33]. Due to space constraints, we only present CodeBERT-ReCall (CBS-R) results, as the result for CodeBERT-Precision, CodeBERT-F1, and CodeBERT-F3 are similar to CodeBERT-ReCall. Please note that, in our experiments, a higher Edit Distance indicates that less modification is required.

Training Settings. Experiments are conducted on a server with a 64-core Intel Xeon Gold CPU and 10 NVIDIA Tesla V100 GPUs (16GB each). We fine-tune CodeLLaMA-7B-Instruct using LoRA [13] with rank r = 32, scaling factor α = 16, dropout rate 0.05, and a maximum token length of 2,048.

B. Main Results

1) Enhancement of Mainstream LLMs: We evaluated TeSyn's effectiveness using five LLMs under the two scenarios from Sec. V-A. We measured the average accuracy of generated functions for each system software using three metrics. As shown in Table II, incorporating TeSyn-synthesized templates into prompts improved all five LLMs' performance across all metrics. Additionally, fine-tuning CodeLLaMA-7B-Instruct on SysRetar to create **SysRetar-LLM** further increased accuracy across all software and metrics by integrating TeSyn-synthesized templates into the training data.

Answer to RQ.1

The code generation accuracy of existing LLMs for system software retargeting has significant room for improvement. Using TeSyn-constructed prompts and fine-tuning can effectively enhance their accuracy.

2) Compare with Fork-Flow: We compared traditional retargeting methods (Fork-Flow) with SysRetar-LLM (-w TeSyn). Fork-Flow, commonly used in manual retargeting, involves copying and modifying functions with identical names from existing targets. To simulate the Fork-Flow process, we calculated three metrics for each inherited function in the test set based on its corresponding function in the train/validation set. We then compared the average and maximum values of these metrics to those of SysRetar-LLM-generated functions. As shown in Fig. 5, SysRetar-LLM (-w TeSyn) surpasses Fork-Flow's maximum values across all system software, highlighting the efficiency of fine-tuned LLMs in retargeting.

Answer to RQ.2

With TeSyn-constructed prompts and fine-tuning, SysRetar-LLM outperforms traditional manual retargeting methods (Fork-Flow) in code generation accuracy, greatly improving system software retargeting efficiency.

C. Ablation Studies

Code Clustering. We constructed a set of prompts using directly synthesized templates from all inherited functions (without code clustering) and fine-tuned CodeLLaMA on this data. The results, shown in Table III, indicate maintained or decreased accuracy across all targets. Further analysis suggests that the lack of code clustering led to overly verbose templates, reducing accuracy.

Dataset Split Scheme. We split the dataset into training (90%) and validation (10%) sets, resulting in 11,491 training prompts and 1,308 validation prompts. Fine-tuning CodeLLaMA with this split showed stable or decreased accuracy across most targets, except for Edit Distance on NVPTX in LLVM, as detailed in Table III. This suggests that the random split scheme is unsuitable for fine-tuning LLMs, leading us to adopt the function-based scheme described in Sec. IV-C.

VI. Related Work

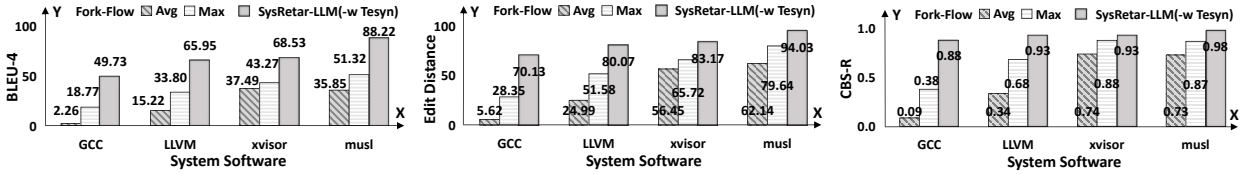
AI for Code Generation. AI-driven code generation tools, particularly those utilizing language models, suggests potential for automating coding tasks [9]–[11], [25], [27], [29]. However, mainstream models still exhibit significant room for improvement in system software retargeting performance. As an effective template synthesis method, TeSyn aims to bridge the gap in LLMs for system software retargeting.

Template Based Retargeting. Template-based retargeting provides an efficient solution characterized by enhanced comprehension, reuse, and scalability [2], [3], [12], [18], [31]. However, these approaches still require developers to learn the associated template rules and write a substantial amount of code, which can be time-consuming and may hinder efficiency.

Authorized licensed use limited to: William & Mary. Downloaded on October 10,2026 at 03:41:57 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Average accuracy of functions generated by selected LLMs across system software in SysRetar.

Models	GCC			LLVM			xvisor			musl		
	BLEU-4	ED	CBS-R	BLEU-4	ED	CBS-R	BLEU-4	ED	CBS-R	BLEU-4	ED	CBS-R
ChatGPT	1.69	29.86	0.60	2.97	37.16	0.72	2.86	32.74	0.65	3.28	37.48	0.59
-w TeSyn	9.56 ↑	46.84 ↑	0.77 ↑	8.47 ↑	48.47 ↑	0.83 ↑	18.85 ↑	54.74 ↑	0.88 ↑	18.38 ↑	52.66 ↑	0.89 ↑
DeepSeek-Coder	2.09	24.97	0.62	2.44	25.93	0.65	2.52	29.15	0.65	3.42	25.55	0.61
-w TeSyn	12.73 ↑	50.01 ↑	0.77 ↑	6.70 ↑	49.86 ↑	0.83 ↑	13.28 ↑	56.89 ↑	0.88 ↑	18.16 ↑	51.93 ↑	0.70 ↑
Granite-Code	1.60	21.86	0.62	2.65	26.53	0.66	2.37	26.96	0.63	4.58	29.12	0.64
-w TeSyn	16.26 ↑	51.27 ↑	0.81 ↑	15.15 ↑	49.61 ↑	0.84 ↑	42.80 ↑	67.00 ↑	0.90 ↑	49.57 ↑	72.00 ↑	0.92 ↑
CodeGemma	1.84	24.83	0.64	1.60	18.90	0.49	2.58	26.24	0.64	2.68	20.91	0.51
-w TeSyn	27.86 ↑	55.40 ↑	0.84 ↑	11.50 ↑	52.68 ↑	0.85 ↑	33.62 ↑	58.39 ↑	0.89 ↑	38.98 ↑	68.36 ↑	0.92 ↑
Code-LLaMA	2.40	34.52	0.68	4.15	35.46	0.74	2.93	34.13	0.73	2.53	42.36	0.71
-w TeSyn	5.50 ↑	51.37 ↑	0.80 ↑	6.84 ↑	50.75 ↑	0.84 ↑	7.85 ↑	59.30 ↑	0.88 ↑	12.32 ↑	57.21 ↑	0.78 ↑
SysRetar-LLM	33.06	60.14	0.83	58.68	75.73	0.90	60.84	78.87	0.90	78.38	89.39	0.96
-w TeSyn	**49.73** ↑	**70.13** ↑	**0.88** ↑	**65.95** ↑	**80.07** ↑	**0.93** ↑	**68.53** ↑	**83.17** ↑	**0.93** ↑	**88.22** ↑	**94.03** ↑	**0.98** ↑



(a) BLEU-4 (b) Edit Distance (c) CodeBERTScore-ReCall

Fig. 5: Comparison of SysRetar-LLM (-w TeSyn) and Fork-Flow.

TABLE III: Results of ablation studies.

Models	GCC			LLVM			xvisor			musl		
	BLEU-4	ED	CBS-R	BLEU-4	ED	CBS-R	BLEU-4	ED	CBS-R	BLEU-4	ED	CBS-R
SysRetar-LLM	33.06	60.14	0.83	58.68	75.73	0.90	60.84	78.87	0.90	78.38	89.39	0.96
-w TeSyn	**49.73**	**70.13**	**0.88**	**65.95**	**80.07**	**0.93**	**68.53**	**83.17**	**0.93**	**88.22**	**94.03**	**0.98**
-w/o code clustering	46.89 ↓	68.15 ↓	0.86 ↓	62.72 ↓	78.84 ↓	0.92 ↓	67.73 ↓	83.09 ↓	0.93 -	83.83 ↓	92.39 ↓	0.98 -
random split	47.40 ↓	69.12 ↓	0.86 ↓	61.55 ↓	78.23 ↓	0.92 ↓	68.31 ↓	83.24 ↑	0.93 -	81.69 ↓	90.71 ↓	0.98 -

In contrast, TeSyn integrates template techniques with LLMs, effectively reducing the manual code-writing overhead.

VII. Limitations

Function Accuracy. Most TeSyn-generated functions still require manual modifications and do not run correctly out-of-the-box. We plan to introduce a program repair mechanism to automatically help fix bugs in generated functions.

Function Coverage. Since TeSyn relies on implementations of the same inherited functions across multiple targets, it is not suited for customized functions unique to a single target. In the future, we aim to introduce a mechanism that matches functions based on similarities and functionality, allowing the synthesis of unified templates for customized functions.

VIII. Conclusion & Future Plans

We present TeSyn, a framework to enhance LLMs' system software retargeting capabilities, and introduce SysRetar, the first dataset for this task. Experiments on five LLMs demonstrate significant performance improvements with TeSyn. Fine-tuning CodeLLaMA on SysRetar yields SysRetar-LLM, which surpasses the traditional Fork-Flow approach in accuracy, promising gains in speed and efficiency. Based on these results, we propose three directions to further advance LLM-driven system software retargeting.

Accuracy Improvement. While SysRetar-LLM (-w TeSyn) generates more accurate code than Fork-Flow, manual checks are still required for successful compilation. To further enhance accuracy, we plan to improve template and code generation using program synthesis [15], [16]. By combining example-based synthesis with LLM-driven techniques, we aim to automatically synthesize templates and code, tailored to the hardware characteristics of different targets.

Error-Correction Mechanism. LLM-generated functions cannot be directly compiled into executable software, complicating debugging. To address this, we will develop an automatic error correction mechanism using program repair [1], [7], [14]. Static analysis tools will detect errors, and an LLM trained on commit history bug-fixes will perform automatic program repair, improving debugging efficiency.

Function Coverage. As discussed in Sec. VII, customized functions are unique to single target, making template synthesis difficult. We plan to use edit distance for name matching to synthesize templates for customized functions with similar names, enabling code generation for these functions.

Acknowledgment

We would like to thank all anonymous reviewers for their insightful feedback. This work was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDB0500100. It was also supported by the National Natural Science Foundation of China, Grant No. U23B2020, No. 62302479, No. 62232015 and the Innovation Funding of ICT, CAS under Grant No. E361010 and No. E261110.

Authorized licensed use limited to: William & Mary. Downloaded on October 10,2026 at 03:41:57 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin Vechev. Tfix: Learning to fix coding errors with a text-to-text transformer. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 780–791. PMLR, 18–24 Jul 2021.

[2] Florian Brandner, Viktor Pavlu, and Andreas Krall. Automatic generation of compiler backends. *Software: Practice and Experience*, 43(2):207–240, 2013.

[3] Gunnar Braun, Achim Nohl, Weihua Sheng, Jianjiang Ceng, Manuel Hohenauer, Hanno Scharwächter, Rainer Leupers, and Heinrich Meyr. A novel approach for flexible and consistent adl-driven asip design. In *Proceedings of the 41st Annual Design Automation Conference*, DAC '04, page 717–722, New York, NY, USA, 2004. Association for Computing Machinery.

[4] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T. Devanbu, and Baishakhi Ray. Natgen: generative pre-training by "naturalizing" source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, page 18–30, New York, NY, USA, 2022. Association for Computing Machinery.

[5] M.A. Ertl and D. Gregg. Retargeting jit compilers by using c-compiler generated executable code. In *Proceedings. 13th International Conference on Parallel Architecture and Compilation Techniques, 2004. PACT 2004.*, pages 41–50, 2004.

[6] Brendan J. Frey and Delbert Dueck. Clustering by passing messages between data points. *Science*, 315(5814):972–976, 2007.

[7] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. Vulrepair: a t5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, page 935–947, New York, NY, USA, 2022. Association for Computing Machinery.

[8] GCC. Gnu compiler collection. https://gcc.gnu.org, 2023.

[9] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7212–7225, Dublin, Ireland, May 2022. Association for Computational Linguistics.

[10] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. Graphcodebert: Pre-training code representations with data flow. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, Virtual Event, Austria, 2021. OpenReview.net.

[11] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024.

[12] Ashok Halambi, Peter Grun, Vijay Ganesh, Asheesh Khare, Nikil Dutt, and Alex Nicolau. Expression: A language for architecture exploration through compiler/simulator retargetability. In *Proceedings of the Conference on Design, Automation and Test in Europe*, DATE '99, page 100–es, New York, NY, USA, 1999. Association for Computing Machinery.

[13] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[14] Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. An empirical study on fine-tuning large language models of code for automated program repair. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1162–1174, 2023.

[15] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models, 2023.

[16] Yixuan Li, Julian Parsert, and Elizabeth Polgreen. Guiding enumerative program synthesis with large language models, 2024.

[17] LLVM. The llvm compiler infrastructure project. http://llvm.org/, 2023.

[18] P. Marwedel. The mimola design system: Tools for the design of digital processors. In *21st Design Automation Conference Proceedings*, pages 587–593, Albuquerque, NM, USA, 1984. IEEE Computer Society.

[19] Mayank Mishra, Matt Stallone, Gaoyuan Zhang, Yikang Shen, Aditya Prasad, Adriana Meza Soria, Michele Merler, Parameswaran Selvam, Saptha Surendran, Shivdeep Singh, Manish Sethi, Xuan-Hong Dang, Pengyuan Li, Kun-Lung Wu, Syed Zawad, Andrew Coleman, Matthew White, Mark Lewis, Raju Pavuluri, Yan Koyfman, Boris Lublinsky, Maximilien de Bayser, Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Yi Zhou, Chris Johnson, Aanchal Goyal, Hima Patel, Yousaf Shah, Petros Zerfos, Heiko Ludwig, Asim Munawar, Maxwell Crouse, Pavan Kapanipathi, Shweta Salaria, Bob Calio, Sophia Wen, Seetharami Seelam, Brian Belgodere, Carlos Fonseca, Amith Singhee, Nirmit Desai, David D. Cox, Ruchir Puri, and Rameswar Panda. Granite code models: A family of open foundation models for code intelligence, 2024.

[20] musl. musl - an implementation of the standard library for linux-based systems. https://git.musl-libc.org/cgit/musl, 2024.

[21] OpenAI. Chatgpt: Optimizing language models for dialogue. https://openai.com/blog/chatgpt/, 2022.

[22] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ACL '02, page 311–318, USA, 2002. Association for Computational Linguistics.

[23] Python. difflib — helpers for computing deltas. https://docs.python.org/3/library/difflib.html, 2023.

[24] riscvarchive. Risc-v software ecosystem overview. https://github.com/riscvarchive/riscv-software-list, 2024.

[25] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open Foundation Models for Code, 2024.

[26] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, page 1433–1443, New York, NY, USA, 2020. Association for Computing Machinery.

[27] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A. Choquette-Choo, Jingyue Shen, Joe Kelley, Kshitij Bansal, Luke Vilnis, Mateo Wirth, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Zhitao Gong, Jane Fine, Tris Warkentin, Ale Jakse Hartman, Bin Ni, Kathy Korevec, Kelly Schaefer, and Scott Huffman. Codegemma: Open code models based on gemma, 2024.

[28] Tree-Sitter. Tree-sitter introduction. https://tree-sitter.github.io/tree-sitter, 2024.

[29] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. CodeT5+: Open Code Large Language Models for Code Understanding and Generation, 2023.

[30] xvisor. Xvisor an open-source bare-metal monolithic hypervisor. https://xhypervisor.org/, 2024.

[31] Drew Zagieboylo, Charles Sherk, Gookwon Edward Suh, and Andrew C. Myers. Pdl: A high-level hardware design language for pipelined processors. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, page 719–732, New York, NY, USA, 2022. Association for Computing Machinery.

[32] Jiacheng Zhao, Yisong Chang, Denghui Li, Chunwei Xia, Huimin Cui, Ke Zhang, and Xiaobing Feng. On retargeting the ai programming framework to new hardwares. In Feng Zhang, Jidong Zhai, Marc Snir, Hai Jin, Hironori Kasahara, and Mateo Valero, editors, *Network and Parallel Computing*, pages 39–51, Cham, 2018. Springer International Publishing.

[33] Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. CodeBERTScore: Evaluating code generation with pretrained models of code. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13921–13937, Singapore, December 2023. Association for Computational Linguistics.

Authorized licensed use limited to: William & Mary. Downloaded on October 10,2026 at 03:41:57 UTC from IEEE Xplore. Restrictions apply.